

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL3-Reverse Engineering

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	Venganapalli Sireesha
Name	192472208

COURSE OUTCOME COVERED

CO5: Design intelligent agents and real-time AI-based systems (chatbots, expert systems, emotion detection, edge AI, autonomous drones, and related applications) by selecting appropriate agent architectures, knowledge representation, and reasoning mechanisms to solve real-world problems. (BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1. The following is a conversation between a user and an AI-based flight-booking chatbot. Study the transcript given below and reverse-engineer the chatbot's design by: (a) writing its PEAS description, (b) classifying the type of agent it represents with justification, (c) identifying the likely component (NLU, Dialogue Manager, Knowledge Base, or NLG) responsible for each chatbot response, and (d) suggesting one enhancement using a learning-agent architecture.

Speaker	Message
User	I want to book a flight to Chennai next Friday.
Chatbot	Sure! What time would you like to depart, and do you have a preferred airline?
User	Morning flight, no preference.
Chatbot	I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?

Q2. The block diagram below shows the processing pipeline of a Real-Time Emotion Detection System. Reverse-engineer the system by: (a) explaining the function performed at each stage of the pipeline, (b) identifying the likely AI/ML technique used at each stage, (c) writing the PEAS description of the system, and (d)

Real-Time Emotion Detection System — processing pipeline



classifying the agent type (e.g., simple reflex, model-based) with justification.

Q3. An Expert System Shell for medical diagnosis contains the rule base, input facts, and reasoning trace given below. Study it and reverse-engineer the shell by: (a) identifying the type of chaining (forward/backward) used to reach the diagnosis, with justification, (b) identifying the expert-system-shell components illustrated (knowledge base, inference engine, explanation facility), (c) explaining how the shell would perform knowledge acquisition to add a new rule for dengue, and (d) explaining, with reasoning, whether backward chaining could reach the same diagnosis.

Rule	IF (condition)	THEN (conclusion)
R1	fever = yes AND cough = yes	suspect = flu
R2	suspect = flu AND body_ache = yes	diagnosis = Influenza
R3	fever = yes AND rash = yes	suspect = measles

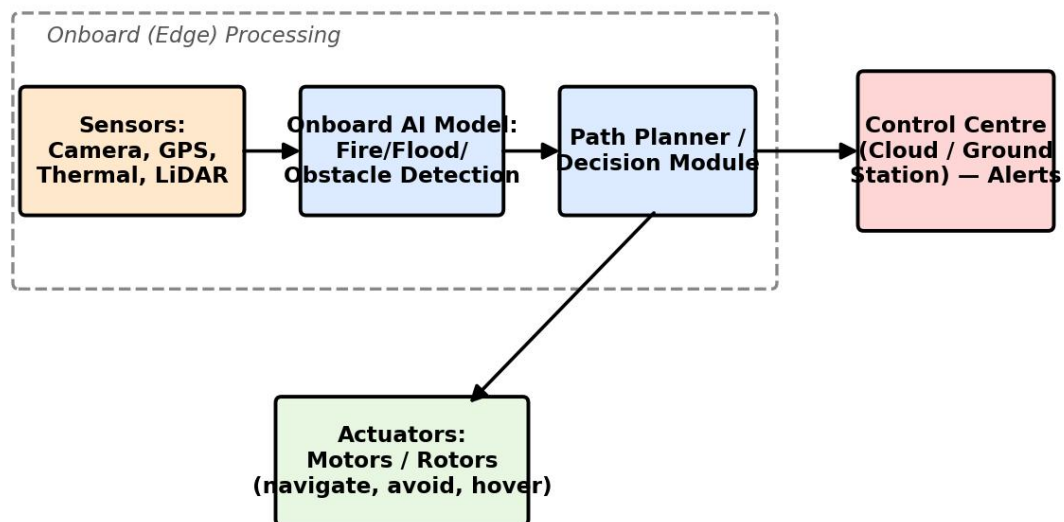
Given facts: fever = yes, cough = yes, body_ache = yes

Reasoning trace: R1 fires first (fever & cough matched) → suspect = flu. R2 then fires (suspect = flu & body_ache matched) → diagnosis = Influenza.

Explanation generated by the shell: “Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza.”

Q4. The diagram below shows the system architecture of an Autonomous Drone used for Disaster Detection with Edge AI deployment. Reverse-engineer the design by: (a) writing the PEAS description of the drone agent, (b) classifying the agent type (goal-based/utility-based) with justification, (c) identifying which components run on the edge versus the cloud and justifying why on-edge processing was chosen for detection and obstacle avoidance, and (d) identifying whether the agent's control architecture is reactive, deliberative, or hybrid, with justification.

Autonomous Drone — Disaster Detection (Edge AI) system architecture



Q5. The table below shows sample output logs from an AI-based Deepfake & Face Authenticity Verification system across five video frames. Study the output and reverse-engineer the system by: (a) identifying the likely input features used by the model (e.g., blink rate, texture/artifact score), (b) identifying the type of model/technique likely used to produce these outputs, (c) explaining how the final Real/Fake decision may have been derived from the frame-level confidence scores,

and (d) suggesting one way a swarm/ensemble-based approach could improve the system's reliability.

Frame No.	Blink Rate (per 10s)	Texture/Artifact Score	Fake Confidence (%)	Prediction
1	4	0.12	8	Real
2	1	0.61	74	Fake
3	0	0.83	91	Fake
4	5	0.09	5	Real
5	2	0.55	68	Fake

Overall system verdict: Fake (3 of 5 frames flagged — majority vote)

Code of Conduct:

I V.Sireesha (192472208) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V.Sireesha

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Identification of Agent Type & PEAS (correctly inferred from the given artifact)	Correctly identifies the agent type and gives a complete, accurate PEAS description, fully consistent with the given artifact.	Identifies the agent type and PEAS description correctly with only minor omissions.	Partially correct identification of agent type/PEAS; some elements are missing or inconsistent with the artifact.	Agent type/PEAS identification is largely incorrect or not attempted.	10
Identification of Architecture & Components (mapping artifact evidence to system components)	Accurately identifies all relevant architectural components and correctly maps each to specific evidence in the artifact.	Identifies most components correctly with reasonable mapping to the artifact; minor gaps present.	Identifies some components; mapping to the artifact is weak or partially incorrect.	Little or no correct identification of architecture/components.	10
Identification of Underlying Technique/Algorithm	Correctly reverse-engineers the	Correctly identifies the	Identification is only partially	Technique/mechanism is incorrectly	10

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Algorithm/Reasoning Mechanism	technique, algorithm, or reasoning mechanism (e.g., chaining type, ML technique) with clear supporting evidence.	technique/mechanism with adequate, mostly accurate justification.	correct or lacks sufficient supporting evidence.	identified or not addressed.	
Quality of Justification & Use of Evidence (linking conclusions back to the given artifact)	Every conclusion is clearly justified with specific, accurate references to the given artifact.	Most conclusions are justified with reasonable references to the artifact.	Justification is vague or only loosely connected to the artifact.	Conclusions are stated with little or no justification or evidence.	10
Clarity & Completeness of Presentation (addresses all sub-parts of the question in an organised manner)	All sub-parts are addressed completely, in a clear, well-organised, and logically structured manner.	Most sub-parts are addressed clearly with minor gaps in organisation or completeness.	Some sub-parts are missing or the presentation is unclear/poorly organised.	Response is incomplete, disorganised, or largely missing.	10
				Total	50

Q1. AI-Based Flight-Booking Chatbot

The given conversation shows a chatbot that understands a flight-booking request, asks for missing information, searches available flights, and recommends an option.

(a) PEAS Description

PEAS Element	Description
Performance Measure	Correct flight booking, accurate understanding of user requirements, relevant flight recommendations, minimum booking errors, fast response, and user satisfaction.
Environment	Flight-booking platform, airline databases, flight schedules, prices, availability, and the user interacting through the chatbot.
Actuators	Chatbot messages, asking clarification questions, displaying available flights, recommending flights, and eventually confirming a booking.
Sensors	User's text messages, destination, travel date, preferred time, airline preference, and flight database information.

(b) Type of Agent

The chatbot can be classified primarily as a model-based goal-based agent.

- Its goal is to help the user find/book a suitable flight.
- It maintains information about the current booking request.
- It asks questions when information is missing.
- It uses the user's answers to select appropriate flights.
- For example, after the user says "Morning flight, no preference," the chatbot identifies three suitable morning flights.

It is more than a simple reflex agent because its response depends on the conversation context and the information already collected.

(c) Component Responsible for Each Response

Chatbot Response	Component	Reason
"Sure! What time would you like to depart, and do you have a preferred airline?"	Dialogue Manager + NLU	NLU extracts the destination/date from the user's request, while the Dialogue Manager identifies missing information and asks a follow-up question.
"I found 3 morning flights to Chennai on Friday."	Knowledge Base + Dialogue Manager	The Knowledge Base provides flight availability, while the Dialogue Manager uses the information to construct the appropriate response.
"Shall I show you the cheapest option first?"	Dialogue Manager + Knowledge Base + NLG	The Knowledge Base supplies flight/pricing information, the Dialogue Manager decides what to suggest, and NLG converts the decision into natural language.

(d) Enhancement Using a Learning-Agent Architecture

A learning agent can be added to improve the chatbot continuously. The learning component could analyse previous booking conversations, user preferences, selected flights, cancellations, and feedback. Over time, it could learn that particular users prefer factors such as lowest price, shortest travel time, preferred airlines, or morning flights.

A possible architecture would contain:

Performance Element → Critic → Learning Element → Problem Generator

For example, if users frequently reject the cheapest flight because of long travel time, the system can learn to recommend flights using a combination of price and travel duration rather than price alone.

Q2. Real-Time Emotion Detection System

The question describes a processing pipeline for a real-time emotion detection system and asks for the function, AI/ML technique, PEAS description, and agent classification.

(a) Function of Each Stage

A typical reverse-engineered pipeline can be represented as:

Input → Preprocessing → Feature Extraction → Emotion Classification → Decision/Output

Stage	Function
Input/Acquisition	Captures real-time facial or audiovisual data from a camera/sensor.
Preprocessing	Removes noise and prepares the captured data for analysis. For facial input, this can include face detection, resizing, and normalization.
Feature Extraction	Extracts important characteristics related to emotion, such as facial landmarks, eye/mouth movements, or learned visual features.
Emotion Classification	Uses the extracted features to determine the emotional state, such as happy, sad, angry, surprised, or neutral.
Decision/Output	Produces the detected emotion and may trigger an appropriate response from the system.

(b) Likely AI/ML Technique

Stage	Likely Technique
Input detection	Computer vision
Preprocessing	Image processing and normalization
Feature extraction	CNN/deep feature extraction or facial landmark extraction
Emotion classification	CNN, deep neural network, or another supervised ML classifier
Final decision	Classification and confidence-based decision

A CNN-based deep-learning model is a strong choice for facial emotion classification because it can automatically learn visual patterns from facial images rather than depending entirely on manually designed features.

(c) PEAS Description

PEAS Element	Description
Performance Measure	High emotion-recognition accuracy, low false detections, fast response, robustness to lighting and facial variations, and real-time performance.
Environment	Real-world surroundings containing people whose facial expressions are being monitored.
Actuators	Displaying the detected emotion, generating alerts, changing system responses, or sending emotion information to another application.
Sensors	Camera/video stream and possibly microphone or other sensors if multimodal emotion detection is used.

(d) Agent Type

The system can be classified as a model-based agent with reactive characteristics.

It continuously receives real-time sensor information and responds to the detected emotional state. However, because emotion recognition involves processing observations and maintaining an interpretation of the person's current state, it is more appropriate to view it as model-based rather than a pure simple-reflex agent.

For example:

Camera input → Facial features → Emotion model → Detected emotion → System response

The real-time nature of the system supports reactive behaviour, while the internal learned model used for emotion recognition supports the model-based classification.

Q3. Medical Diagnosis Expert System

The given system contains rules R1–R3, input facts, a reasoning trace, and an explanation facility. The facts are fever = yes, cough = yes, body_ache = yes. R1 first derives suspect = flu, and R2 then derives diagnosis = Influenza.

(a) Type of Chaining

The system uses forward chaining.

The evidence is directly visible in the reasoning trace:

Initial facts:

- fever = yes
- cough = yes
- body_ache = yes

Step 1:

R1:

fever = yes AND cough = yes → suspect = flu

Therefore:

suspect = flu

Step 2:

R2:

suspect = flu AND body_ache = yes $\rightarrow$ diagnosis = Influenza

Therefore:

diagnosis = Influenza

This is forward chaining because the inference starts with known facts and repeatedly applies rules to derive new facts until the final diagnosis is reached.

(b) Expert-System Shell Components

Component	Evidence/Function
Knowledge Base	Contains R1, R2, and R3, which represent medical knowledge in IF–THEN form.
Inference Engine	Matches the given facts against rule conditions and fires R1 followed by R2.
Explanation Facility	Produces the explanation: the diagnosis was obtained because fever and cough indicated flu, and flu together with body ache confirmed Influenza.
Working Memory/Input Facts	Stores the current patient facts: fever, cough, and body ache.

Thus, the artifact demonstrates the major components of an expert-system shell.

(c) Knowledge Acquisition for a New Dengue Rule

To add a dengue rule, the system developer would first obtain knowledge from a medical expert or validated medical knowledge source.

For example, suppose the expert provides:

IF fever = yes AND rash = yes AND body_ache = yes

THEN suspect = dengue

The knowledge-acquisition process would be:

1. Identify dengue-related symptoms from the domain expert.
2. Convert the expert's knowledge into IF–THEN rules.
3. Validate the rule with the expert.
4. Add the rule to the Knowledge Base.
5. Test it using suitable patient facts.
6. Check whether the inference engine correctly fires the new rule.
7. Ensure the explanation facility can explain the resulting diagnosis.

The new rule should not simply be added without validation because incorrect medical rules can lead to incorrect diagnoses.

(d) Can Backward Chaining Reach the Same Diagnosis?

Yes, backward chaining can reach the same diagnosis.

The goal is: diagnosis = Influenza

The system searches for a rule that can produce this conclusion.

R2 produces Influenza:

suspect = flu AND body_ache = yes $\rightarrow$ diagnosis = Influenza

Therefore, the system needs to prove:

- suspect = flu
- body_ache = yes

The fact body_ache = yes is already available.

To prove suspect = flu, the system examines R1: fever = yes AND cough = yes $\rightarrow$ suspect = flu

Both fever and cough are given facts.

Therefore: fever + cough $\rightarrow$ flu $\rightarrow$ Influenza

Hence, backward chaining can obtain the same diagnosis, but it starts from the goal and works backward toward the supporting facts, unlike the given forward-chaining trace.

Q4. Autonomous Drone for Disaster Detection with Edge AI

The question describes an autonomous drone designed for disaster detection and asks for PEAS, agent type, edge/cloud responsibilities, and control architecture.

(a) PEAS Description

PEAS Element	Description
Performance Measure	Accurate disaster detection, safe navigation, effective obstacle avoidance, minimum collision risk, timely detection, efficient energy usage, and successful completion of the mission.
Environment	Disaster-affected areas containing buildings, people, debris, obstacles, smoke, damaged infrastructure, and changing environmental conditions.
Actuators	Drone motors/rotors, flight-control mechanisms, camera orientation, navigation controls, and communication mechanisms.
Sensors	Cameras, LiDAR/depth sensors, GPS, IMU, ultrasonic sensors, and other onboard sensors.

(b) Agent Type: Goal-Based or Utility-Based

The drone is best classified as a utility-based agent.

Its main goal is disaster detection, but it must also make decisions by balancing several factors:

- Detection accuracy
- Flight safety
- Obstacle avoidance
- Battery/energy consumption
- Mission completion time
- Coverage of the disaster area

A goal-based agent only asks whether the goal is achieved. A utility-based agent can compare different possible actions and select the one that provides the highest overall utility.

For example, when two routes can reach the same location, the drone can select the route that is safer and consumes less battery.

(c) Edge vs Cloud Processing

Processing Location	Components/Tasks	Reason
Edge/On-drone	Camera processing, object/person detection, obstacle detection, immediate navigation, emergency decisions	Requires very low latency and must continue even when network connectivity is poor.
Cloud	Large-scale data storage, detailed analysis, model training, mission reports, historical analysis	Cloud computing provides greater computational and storage resources for non-immediate tasks.

Why detection and obstacle avoidance should run on the edge:

Obstacle avoidance is a time-critical operation. If the drone sends camera data to the cloud, waits for processing, and then receives a command, network delay could cause a collision.

Therefore: Sensor → Edge AI → Immediate decision → Motor/control action
is safer than: Sensor → Cloud → Network delay → Decision → Drone
Edge AI also allows the drone to operate during weak or unavailable network connectivity.

(d) Control Architecture

The drone uses a hybrid control architecture.

Reactive component:

- Quickly responds to immediate obstacles.
- Uses real-time sensor data.
- Performs emergency avoidance.

Deliberative component:

- Plans the mission route.
- Determines areas to search.
- Uses higher-level reasoning to achieve the disaster-detection objective.

Therefore, combining reactive obstacle avoidance with deliberative mission planning makes the architecture hybrid.

Q5. Deepfake & Face Authenticity Verification

The system provides five frame-level outputs containing blink rate, texture/artifact score, fake confidence, and prediction. The overall verdict is Fake because 3 of 5 frames were classified as Fake by majority voting.

(a) Likely Input Features

The table explicitly provides two important features:

1. Blink Rate
2. Texture/Artifact Score

The frame-level data are:

Frame	Blink Rate	Texture/Artifact Score	Fake Confidence	Prediction
1	4	0.12	8%	Real
2	1	0.61	74%	Fake
3	0	0.83	91%	Fake
4	5	0.09	5%	Real
5	2	0.55	68%	Fake

These values are directly given in the artifact.

The model may therefore use:

- Abnormal blink patterns
- Facial texture inconsistencies
- Compression/artifact patterns
- Temporal facial changes
- Facial landmark movement
- Other learned visual features

(b) Likely Model/Technique

The system most likely uses a supervised deep-learning classification model.

A suitable implementation could use:

- CNN for spatial facial features
- CNN + LSTM/temporal model for changes across video frames
- A binary classifier for Real/Fake
- A confidence score representing the model's probability or confidence that a frame is fake

The strong relationship in the table supports this interpretation. For example, Frame 3 has a high texture/artifact score of 0.83 and a high Fake Confidence of 91%, while Frame 4 has a low artifact score of 0.09 and only 5% Fake Confidence.

(c) Deriving the Final Real/Fake Decision

The system appears to perform frame-level classification followed by majority voting.

From the five frames:

- Frame 1 → Real
- Frame 2 → Fake
- Frame 3 → Fake
- Frame 4 → Real
- Frame 5 → Fake

Therefore:

Real = 2 frames

Fake = 3 frames

Since Fake has the majority: 3 Fake > 2 Real

the final system verdict is: Overall Verdict = FAKE

This is explicitly stated in the given artifact as "Fake (3 of 5 frames flagged — majority vote)."

The confidence scores further support the decision because Frames 2, 3, and 5 have relatively high Fake Confidence values of 74%, 91%, and 68%, respectively.

(d) Swarm/Ensemble-Based Improvement

The reliability can be improved by using an ensemble of multiple deepfake detection models instead of depending on a single model.

For example:

Model 1: CNN-based spatial artifact detector

Model 2: Temporal model for facial movements

Model 3: Blink/landmark-based detector

Model 4: Frequency/artifact detector

Each model produces a Real/Fake prediction and confidence score. A swarm/ensemble decision mechanism can then combine these predictions.

For example:

Final Score = weighted combination of all model outputs

If most independent models identify a video as Fake, the system can produce a high-confidence Fake verdict.

This approach improves reliability because a weakness in one model may be compensated for by the other models. It can also reduce false positives and false negatives compared with relying on a single detector.